# TransitGate — Contrato de la API (v1.1)

Rutas y formato JSON entre el ESP32 principal, el servicio de visión, la web/app y el servidor. Completa el pendiente de `TransitGate_Arquitectura_del_sistema_v2.md` sección 9. Cubre los 46 RF de `Requisitos funcionales (v2)` y usa sus supuestos (S1–S15).

El detalle de las interfaces que no son HTTP (APDU entre la app y el lector, stream de la cámara, pines, `config.json` y simuladores) está en `TransitGate_Contratos_de_integracion.md`.

**Cambios en v1.1** (alineación con los RF v2): la lectura lleva `lecturaId` (RF-18 CA4); el desafío del celular viaja en la respuesta del latido y desaparece `POST /desafios` (S12); el latido confirma la apertura con `pasoIdAbierto` (RF-22 CA4, S15); el cruce lleva `trackId` y la hora es la de llegada del fotograma al servicio de visión (RF-26, S13); se define la respuesta a una lectura en `ModoRegistro`.

---

## 1. Convenciones generales

- **Transporte:** todo por HTTP, dentro de la red local de la demo o de desarrollo (sin HTTPS, sin túnel — ver arquitectura v2, sección 9).
- **Formato:** `Content-Type: application/json` en cuerpo y respuesta, salvo el stream MJPEG de la cámara.
- **Fechas:** UTC en formato ISO-8601 (`2026-09-27T20:30:00Z`); el cliente las muestra en hora de Colombia (RNF-42).
- **Errores de validación (400):**

```json
{ "error": "SaldoInsuficiente", "mensaje": "El saldo no alcanza para esta recarg
```

El campo `error` es un código estable (para el código); `mensaje` es texto en español claro para mostrar (RNF-32).

- **Códigos de autorización (RNF-19, decisión #16):**
  - `401` — sin sesión o sin credencial de dispositivo.
  - `403` — sesión válida pero el rol no tiene permiso.
  - `404` — el recurso existe pero es de otro cliente, o no existe (mismo código para no revelar cuál de los dos).
- **Límite de intentos de login:** 6 intentos fallidos bloquean el login temporalmente (RNF-22, RF-02 CA3).

## 2. Autenticación

| Canal | Mecanismo | Vence |
|---|---|---|
| Portal web (cliente, operador, seguridad, administrador) | Cookie de sesión de ASP.NET Core Identity | 60 min de inactividad o menos (RNF-18) |
| App Android | Token de vida corta en `Authorization: Bearer …`, solo para login y vinculación; la emulación NFC no depende de él (RF-19 CA6) | 15 min o menos (RNF-18) |
| ESP32 principal / ESP32-CAM / servicio de visión | Credencial de dispositivo fija por torniquete, header `X-Device-Key` | No vence; es dato semilla desde la Fase 1 (decisión #15) |

```
POST /api/auth/login
{ "correo": "ana@example.com", "password": "••••••••" }

→ 200 { "rol": "Cliente" }              (+ cookie de sesión)
→ 401 { "error": "CredencialesInvalidas", "mensaje": "Correo o contraseña
incorrectos." }
```

```
POST /api/auth/logout
→ 204
```

```
POST /api/auth/recuperar-password
{ "correo": "ana@example.com" }

→ 202                                   (siempre, no revela si el correo
existe; S9)
```

```
POST /api/auth/restablecer-password
{ "token": "…", "passwordNueva": "••••••••" }

→ 200
→ 400 { "error": "TokenVencido", "mensaje": "El enlace venció, solicita uno
nuevo." }   (S9: 30 min)
```

## 3. Clientes (RF-01 a RF-06)

```
POST /api/clientes
{
  "nombreCompleto": "Ana Ríos",
  "tipoDocumento": "CC",
  "numeroDocumento": "1032456789",
  "correo": "ana@example.com",
  "telefono": "3001234567",
  "password": "••••••••",
  "autorizaTratamientoDatos": true
}

→ 201 { "clienteId": "c_9f...", "tarjetaId": "t_1a..." }   (+ sesión iniciada; crea Tarjeta con saldo 0, RF-07)
→ 400 { "error": "CorreoDuplicado", "mensaje": "Ya existe una cuenta con ese correo." }
→ 400 { "error": "DocumentoDuplicado", "mensaje": "Ya existe una cuenta con ese documento." }
→ 400 { "error": "PasswordCorta", "mensaje": "La contraseña debe tener al menos 8 caracteres." }
→ 400 { "error": "AutorizacionRequerida", "mensaje": "Debes autorizar el tratamiento de datos." }   (RNF-28)
```

```
GET  /api/clientes/me          → 200 { "clienteId", "nombreCompleto", "correo", "telefono", "tipoDocumento", "numeroDocumento" }
PATCH /api/clientes/me
{ "telefono": "3009876543" }
→ 200
```

Todos los endpoints de cliente requieren sesión de rol `Cliente` sobre sus propios datos (401 / 403 / 404 según corresponda, RF-06).

## 4. Tarjetas y soportes (RF-07 a RF-14)

```
GET /api/tarjetas/mi-tarjeta
→ 200 {
    "tarjetaId": "t_1a...",
    "saldo": 12000,
    "soporte": { "tipo": "fisico", "estado": "Activa" }   // o "virtual" / null
si aún no tiene soporte
  }
```

```
POST /api/tarjetas/mi-tarjeta/solicitud-fisica            (RF-08)
→ 201 { "solicitudId": "s_44...", "estado": "Solicitada" }
```

Entrega y vinculación en taquilla (RF-09, RF-10; el torniquete pasa al estado `ModoRegistro`, S6):

```
POST /api/torniquetes/{torniqueteId}/modo-registro          Actor: Operador
{ "solicitudId": "s_44..." }
→ 200 { "expiraEn": 60 }
→ 409 { "error": "TorniqueteOcupado", "mensaje": "El torniquete no está
disponible para modo registro." }
```

Mientras el torniquete está en `ModoRegistro`, la siguiente lectura que llega por `POST /api/torniquetes/{id}/lecturas` (sección 6) no se valida como un paso: se liga ese UID a la solicitud, la tarjeta pasa a `Entregada`, el torniquete vuelve a `Cerrado` y la respuesta al ESP32 es:

```
→ 200 { "accion": "registrada", "lecturaId": "a1f3-0007" }       (el brazo no se
mueve)
→ 200 { "accion": "denegar", "motivo": "UidYaRegistrado" }        (el UID ya
pertenece a otro soporte)
```

```
POST /api/tarjetas/mi-tarjeta/confirmar-recepcion            (RF-11, S5:
Entregada → Activa)
→ 200 { "estado": "Activa" }
→ 409 { "error": "EstadoInvalido", "mensaje": "La tarjeta no está en estado
Entregada." }
```

```
POST /api/tarjetas/mi-tarjeta/soporte-virtual                    (RF-12, Should)
{ "confirmarReemplazo": false }
→ 201 { "soporteId": "3f9c0d2a-7b41-4e8a-9c55-1d2e3f4a5b6c", "claveSecreta":
"B64 de 32 bytes" }   (solo se entrega una vez; RNF-21)
→ 409 { "error": "SoporteActivoExistente", "mensaje": "Ya tienes un soporte
activo; confirma el reemplazo." }
                                                                  (RF-13/S1:
reenviar con confirmarReemplazo: true
                                                                   bloquea el
soporte anterior con motivo Reemplazo)
```

```
POST /api/tarjetas/mi-tarjeta/bloquear                          (RF-14)   Actor:
Cliente u Operador/Seguridad
{ "motivo": "Perdida" }        // o "Reporte"
→ 200 { "estado": "Bloqueada" }
```

---

## 5. Saldo y recargas (RF-15 a RF-17)

```
POST /api/tarjetas/mi-tarjeta/recargas
{ "monto": 5000 }
→ 201 { "recargaId": "r_88...", "saldoNuevo": 17000 }
→ 400 { "error": "MontoFueraDeRango", "mensaje": "El monto debe estar entre
1.000 y 200.000 COP." }   (S8)
```

```
GET /api/tarjetas/mi-tarjeta/recargas
→ 200 [ { "recargaId": "r_88...", "monto": 5000, "fecha": "2026-09-27T20:30:00Z"
}, … ]
```

---

## 6. Validación de acceso — ESP32 principal (RF-18 a RF-25, RF-45, RF-46)

Autenticado con `X-Device-Key` del torniquete. Timeouts y reintentos del lado del ESP32 en el documento de integración (C1).

### 6.1 Lectura de tarjeta física

`lecturaId` lo genera el ESP32 (contador de arranque + contador de lecturas). Un reintento con el mismo `lecturaId` recibe la misma respuesta y no debita otra vez (RF-18 CA4, RF-46 CA4).

```
POST /api/torniquetes/{id}/lecturas
{ "lecturaId": "a1f3-0007", "tipo": "uid", "valor": "04A1B2C3" }

→ 200 { "accion": "abrir", "pasoId": 123, "saldoRestante": 8800 }        (RF-20,
RF-21, RF-22)
→ 200 { "accion": "denegar", "motivo": "SaldoInsuficiente" }
→ 200 { "accion": "denegar", "motivo": "TarjetaNoRegistrada" }
→ 200 { "accion": "denegar", "motivo": "SoporteNoActivo" }
→ 200 { "accion": "denegar", "motivo": "SoporteBloqueado" }
→ 200 { "accion": "denegar", "motivo": "TorniqueteOcupado" }
→ 200 { "accion": "denegar", "motivo": "TorniqueteEnAlarma" }
→ 200 { "accion": "registrada" }
(torniquete en ModoRegistro, sección 4)
```

`valor` va en hexadecimal mayúsculas con su longitud real: 8 caracteres (MIFARE Classic, 4 bytes) o 14 (NTAG, 7 bytes) (RNF-35). Motivos completos del catálogo en `Requisitos funcionales (v2)`, sección 1.

### 6.2 Lectura de celular (Should, Fase 4)

El desafío no se pide aparte: llega en cada respuesta del latido (6.3). Cuando el objetivo indica ISO 14443-4, el ESP32 hace el SELECT AID, le pasa a la app el desafío vigente y envía lo que la app responde (formato APDU en el documento de integración, C2):

```
POST /api/torniquetes/{id}/lecturas
{
  "lecturaId": "a1f3-0008",
  "tipo": "celular",
  "soporteId": "3f9c0d2a-7b41-4e8a-9c55-1d2e3f4a5b6c",
  "desafioId": "d_55...",
  "firma": "7A3F...C901"            // HMAC-SHA256 en hex mayúsculas (64
caracteres), ver integración §3
}

→ 200 { "accion": "abrir", "pasoId": 124, "saldoRestante": 5600 }
→ 200 { "accion": "denegar", "motivo": "FirmaInvalida" }                (otra
clave u otro torniquete, RF-19 CA2/CA4)
→ 200 { "accion": "denegar", "motivo": "DesafioInvalido" }              (ya
usado, no lo emitió el servidor o venció, S12)
→ 200 { "accion": "denegar", "motivo": "SoporteBloqueado" }
```

### 6.3 Latido: estado deseado, confirmación y desafío (RF-45, RF-46, S11, S12)

El ESP32 lo llama cada 300 ms. Sirve a la vez de señal de vida (sin conexión tras 5 s sin latidos, S14), de confirmación de apertura (si no llega `pasoIdAbierto` igual al paso dentro de 2 s, el débito se revierte, S15) y de canal del desafío del celular.

```
POST /api/torniquetes/{id}/latido
{
  "brazoReal": "abierto",          // "abierto" | "cerrado"
  "sirenaReal": false,
  "pasoIdAbierto": 123,            // paso para el que abrió el brazo; null si
está cerrado
  "arranque": "a1f3",              // cambia en cada reinicio del ESP32
  "fw": "0.3.0"
}

→ 200 {
    "brazoDeseado": "abierto",
    "sirenaDeseada": false,
    "pasoId": 123,                 // paso pendiente, o null
    "modoRegistro": false,         // el ESP32 puede mostrarlo con el LED
    "desafio": { "desafioId": "d_55...", "valor": "8C1E...4F0A", "expiraEn": 30
}
  }
```

`desafio.valor` son 16 bytes aleatorios en hex (32 caracteres). El servidor puede repetir el mismo desafío en varios latidos mientras no se use ni venza.

Aplicar el mismo estado deseado dos veces no tiene efecto (RF-45 CA2). Si el reportado no coincide con el deseado por más de 2 s, el panel marca el torniquete como desincronizado (RF-45 CA4).

---

## 7. Detección de colados — servicio de visión (RF-26 a RF-30)

Autenticado con `X-Device-Key` del servicio de visión. El servicio corre en la misma laptop que la API, así que comparten reloj y no hace falta sincronizarlos.

```
POST /api/torniquetes/{id}/cruces
{
  "trackId": 4812,                                  // id de seguimiento
(ByteTrack); se reinicia con el servicio
  "sesionVision": "v-20261001-1",                   // cambia en cada arranque
del servicio
  "sentido": "entrada",                             // "entrada" | "salida"
  "horaFotograma": "2026-09-27T20:30:01.450Z"       // llegada del fotograma al
servicio de visión (S13)
}

→ 200 { "resultado": "consumido", "pasoId": 123 }  (había un PasoAutorizado
Pendiente, RF-27)
→ 200 { "resultado": "fraude", "alertaId": "al_9..." }   (cruce sin paso
disponible, RF-28)
→ 200 { "resultado": "registrado" }                (sentido "salida": se
registra, no consume ni es fraude, S7)
→ 200 { "resultado": "duplicado" }                 (mismo sesionVision + trackId
ya reportado, RF-26 CA3)
```

La pareja `sesionVision` + `trackId` es la clave de idempotencia. La regla de la ventana (10 s de S4 más la gracia de S13) la aplica el servidor con `horaFotograma`, no con la hora de llegada a la API.

```
POST /api/torniquetes/{id}/vision/latido                    (cada 5 s; fuera de
línea tras 15 s sin señal, S14)
{ "fps": 14.8, "sesionVision": "v-20261001-1" }
→ 204
```

---

## 8. Monitoreo y operación (RF-31 a RF-38)

Requiere sesión de rol `Operador`, `Seguridad` o `Administrador`, salvo donde se indique.

```
GET /api/torniquetes
→ 200 [ { "id", "nombre", "estado":
"Cerrado|EsperandoCruce|Alarma|ModoRegistro",
          "conectado": true, "ultimaSenal": "…" }, … ]
(RF-31, RF-37)
```

```
GET /api/torniquetes/{id}/intentos?desde=&hasta=
→ 200 [ { "intentoId", "resultado": "Exitoso|Rechazado|Fraude", "motivo",
"fecha" }, … ]   (RF-32)
```

```
GET /api/alertas?estado=Abierta
→ 200 [ { "alertaId", "torniqueteId", "estado": "Abierta|Atendida|Resuelta",
"fecha" }, … ]   (RF-33)
```

```
POST  /api/alertas/{id}/apagar-alarma          → 200
(RF-34)
PATCH /api/alertas/{id}
{ "estado": "Resuelta", "clasificacion": "FraudeConfirmado", "notas": "…" }
→ 200
(RF-35)
```

```
GET /api/tarjetas/solicitudes?estado=Solicitada
→ 200 [ { "solicitudId", "clienteId", "nombreCompleto", "fecha" }, … ]
(RF-36)
```

```
GET /api/torniquetes/{id}/video
(RF-38, Could)
→ 200  (proxy/redirect al stream MJPEG del ESP32-CAM)
```

---

## 9. Historial del cliente (RF-39, RF-40)

```
GET /api/tarjetas/mi-tarjeta/viajes?desde=&hasta=
→ 200 [ { "intentoId", "torniqueteNombre", "tarifa": 3200, "fecha" }, … ]
```

---

## 10. Administración (RF-41 a RF-44)

Requiere sesión de rol `Administrador`.

```
PATCH /api/admin/tarifa
{ "valor": 3200 }
→ 200
```

```
POST  /api/admin/operadores        { "nombreCompleto", "correo", "rol":
"Operador|Seguridad" }   → 201
PATCH /api/admin/operadores/{id}  { "activo": false }
→ 200   (RF-42)
```

```
POST /api/admin/torniquetes
{ "nombre": "Torniquete 1", "ubicacion": "Entrada norte" }
→ 201 { "id": "tq_1", "credencialDispositivo": "…" }            (dato semilla,
decisión #15; RF-43)
```

```
PATCH /api/admin/configuracion
{
  "expiracionPasoSegundos": 10,          // S4
  "duracionModoRegistroSegundos": 60,    // S6
  "montoMinRecarga": 1000,               // S8
  "montoMaxRecarga": 200000,             // S8
  "desafioExpiraSegundos": 30,           // S12
  "plazoSenalEsp32Segundos": 5,          // S14
  "plazoSenalVisionSegundos": 15         // S14
}
→ 200
→ 400 { "error": "ValorFueraDeRango", "mensaje": "La expiración debe estar entre
1 y 60 s." }   (RF-44 CA2)
```

Los valores por defecto viven en `appsettings.json` (servidor) y `config.json` (ESP32); lo que el administrador guarda aquí prevalece (RNF-40).

---

## 11. Tiempo real — SignalR (`/hubs/eventos`)

| Evento | Para quién | Cuándo |
|---|---|---|
| `SaldoActualizado` | Cliente (su propia tarjeta) | Tras una recarga o un paso exitoso |
| `EventoViaje` | Cliente | Tras un `IntentoAcceso` sobre su tarjeta |
| `EstadoTorniquete` | Operador / Seguridad | Cambio de estado (`Cerrado`, `EsperandoCruce`, `Alarma`, `ModoRegistro`) o de conexión |
| `AlertaNueva` / `AlertaResuelta` | Operador / Seguridad | Al crear o resolver una `AlertaSeguridad` |
| `SolicitudNueva` | Operador | Al solicitarse una tarjeta física |

---

## 12. Resumen de endpoints por actor

| Actor | Endpoints |
|---|---|
| Cliente (web) | `/api/clientes`, `/api/auth/*`, `/api/tarjetas/mi-tarjeta*` |
| App Android | `/api/auth/login`, `/api/tarjetas/mi-tarjeta/soporte-virtual` |
| Operador | `/api/torniquetes/{id}/modo-registro`, `/api/tarjetas/solicitudes`, `/api/alertas/*` |
| Seguridad | `/api/alertas/*`, `/api/torniquetes` |
| Administrador | `/api/admin/*` |
| ESP32 principal | `/api/torniquetes/{id}/lecturas`, `/latido` |
| Servicio de visión | `/api/torniquetes/{id}/cruces`, `/vision/latido` |